



دانشگاه صنعتی امیرکبیر
دانشکده مهندسی کامپیوتر



فاز اول پروژه پایانی

تحلیل لاگ جام جهانی با Nginx و MapReduce

مبانی رایانش ابری

نیمسال دوم ۱۴۰۴-۱۴۰۵

گزارش طراحی، پیاده‌سازی و آزمون

مشخصات گروه

سید حسام الدین حسینیان ۴۰۲۳۱۹۰۱

فروزان قویدل ۴۰۲۳۱۰۶۴

استاد درس: دکتر جوادی

تدریس‌یاران: مهرداد شیخ‌عباسی و محمدهادی ستاک

بهار ۱۴۰۵

فهرست مطالب

۱	صورت مسئله و محدوده کار	۲
۲	بررسی کد اولیه و برنامه اجرا	۲
۳	معماری نهایی سامانه	۳
۴	پیاده‌سازی سرویس‌های Backend	۳
۵	تنظیم Nginx	۵
۶	مولد ترافیک	۶
۷	خطاهای مشاهده‌شده و اصلاح آن‌ها	۷
۸	راه‌اندازی Hadoop و HDFS	۹
۹	پیاده‌سازی پنج Job با Streaming Hadoop	۹
۱۰	تحلیل خروجی‌ها	۱۱
۱۱	بخش امتیازی Streaming Structured Spark	۱۳
۱۲	کنترل مصرف CPU و RAM	۱۳
۱۳	آزمون‌ها و معیار پذیرش	۱۴
۱۴	روش اجرای پروژه	۱۴
۱۵	ساختار فایل‌های تحویلی	۱۶
۱۶	جمع‌بندی	۱۶

صورت مسئله و محدوده کار

هدف این فاز، ساخت یک مسیر کامل از تولید درخواست تا تحلیل نهایی لاگ بود. در این مسیر، سه سرویس اطلاعات بازی، تیم و ورزشگاه پشت یک درگاه Nginx قرار گرفتند. ترافیک آزمایشی فقط به Nginx ارسال شد، هر درخواست در دو سطح gateway و backend لاگ شد و در پایان، پنج مرحله پردازش دسته‌ای با Streaming Hadoop اجرا شد.

فایل دستورکار ۲۵ صفحه‌ای ابتدا به‌طور کامل بررسی شد. سپس موارد تحویلی به یک فهرست اجرایی تبدیل شد تا پیاده‌سازی به ترتیب انجام شود و هیچ خروجی اجباری جا نماند. بخش‌های اصلی کار عبارت بودند از:

- تکمیل لاگ ساخت‌یافته در هر سه سرویس پایه؛
- نوشتن Dockerfile مستقل و فایل Compose برای سرویس‌ها؛
- تنظیم Nginx به عنوان تنها ورودی عمومی سامانه؛
- نوشتن generator traffic برای سناریوهای عادی، نامعتبر، کند، پرتراфик و خطای سرور؛
- تولید اجرای نهایی با حداقل ۱۰۰ هزار درخواست؛
- پیاده‌سازی های‌Job ۱ تا ۵ با mapper و reducer پایتون؛
- اجرای واقعی های‌Job با Streaming Hadoop و ذخیره همه خروجی‌های میانی و نهایی؛
- کنترل مصرف پردازنده و حافظه در تمام مراحل.

نتیجه کلی ✓

اجرای نهایی شامل دقیقاً ۱۰۰,۰۰۰ درخواست بود. فایل Nginx دقیقاً ۱۰۰,۰۰۰ رکورد و مجموع سه فایل سرویس نیز ۱۰۰,۰۰۰ رکورد داشت. پنج Job روی HDFS اجرا شدند و validator سیزده خروجی اجباری را بدون خطا تأیید کرد.

بررسی کد اولیه و برنامه اجرا

در نسخه اولیه فقط فایل main.py سه سرویس و Compose نمونه Hadoop وجود داشت. تابع write_service_log در هر سه سرویس خالی بود و Dockerfile پیکربندی Nginx، مولد ترافیک و کد MapReduce وجود نداشت. قالب گزارش نیز فقط شامل ظاهر و صفحه عنوان بود و بدنه‌ای نداشت.

برای کنترل ریسک، اجرا به سه سطح تقسیم شد:

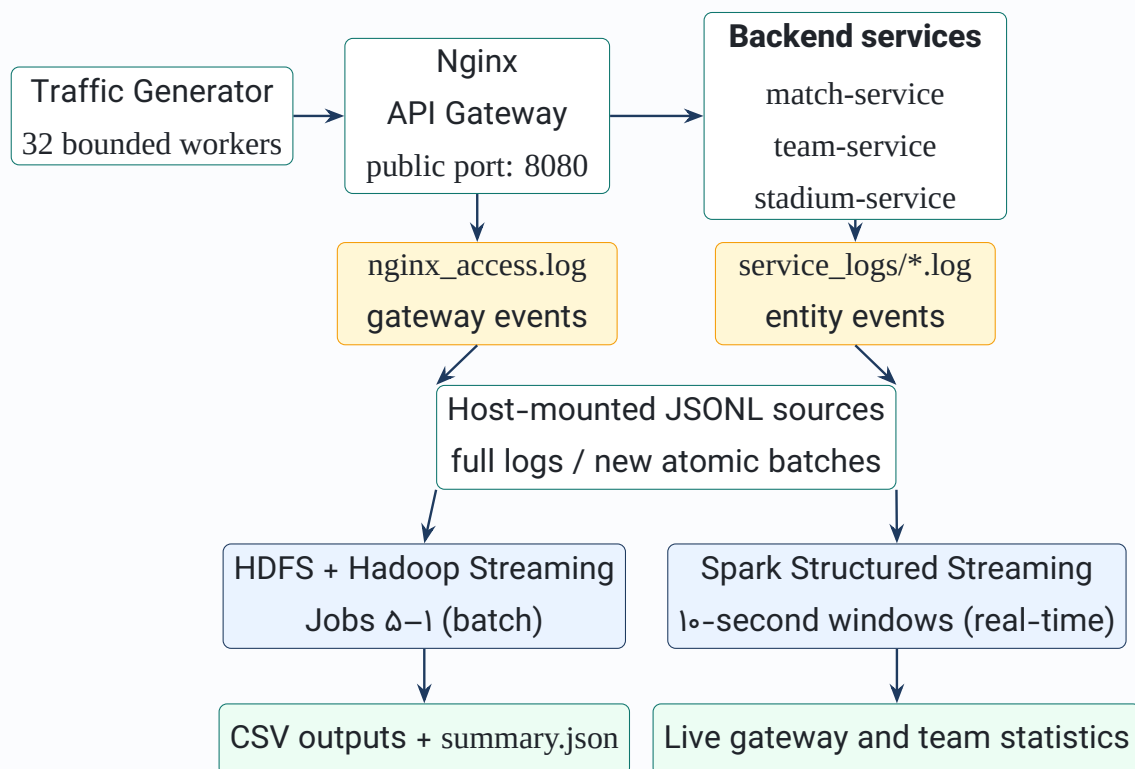
۱. بررسی نحوی و تست مستقیم توابع بدون بالا آوردن سرویس‌های سنگین؛

۲. اجرای Debug با ۱۰۰۰ درخواست و تست محلی همان ها: mapper/reducer

۳. اجرای نهایی ۱۰۰ هزار درخواست و سپس اجرای Hadoop، به صورت جدا از stack سرویس ها.

در ابتدای کار سیستم ۷/۴ گیگابایت RAM و ۱۶ پردازنده منطقی داشت، اما فقط حدود ۲/۹ گیگابایت RAM در دسترس بود. به همین دلیل اجرای API و Hadoop همزمان انجام نشد و برای تمام کانتینرها سقف منبع تعیین شد.

معماری نهایی سامانه



شکل ۱: معماری نهایی سامانه و مسیر جداگانه پردازش batch و real-time

هیچ پورت backend روی میزبان publish نشده است. بنابراین مولد ترافیک از بیرون شبکه Docker راهی برای دور زدن Nginx ندارد و تنها پورت عمومی، 8080 است.

پیاده سازی سرویس های Backend

تکمیل لاگ ساخت یافته

در هر سه سرویس، یک رکورد JSON مستقل برای هر درخواست نوشته می شود. برای جلوگیری از مخلوط شدن خطوط در درخواست های همزمان، نوشتن فایل با threading.Lock محافظت شد. مسیر فایل نیز از

متغیر محیطی خوانده می‌شود تا کد به مسیر مطلق سیستم شخصی وابسته نباشد.

فیلدهای ثبت‌شده در لاگ سرویس‌ها

فیلد	کاربرد
timestamp	زمان UTC با قالب ISO-۸۶۰۱
request_id	شناسه منتقل‌شده از Nginx برای تطبیق دو لاگ
client_country	کشور کاربر آزمایشی
scenario	نوع سناریوی ترافیک؛ این فیلد افزون بر حداقل schema نگه داشته شد
service, endpoint	نام سرویس و مسیر بدون query
entity_type/value	نوع و مقدار تیم، روز مسابقه، ورزشگاه یا شهر
status_code	کد واقعی پاسخ
processing_time_ms	زمان پردازش داخل سرویس
event_type	نوع رخداد جست‌وجو

رفتارهای endpoint اولیه حفظ شد: ورودی معتبر پاسخ ۲۰۰، ورودی ناقص یا با قالب نامعتبر پاسخ ۴۰۰، موجودیت ناموجود پاسخ ۴۰۴ و سناریوی server_error پاسخ ۵۰۰ می‌دهد. سناریوی slow نیز تأخیر کنترل‌شده ایجاد می‌کند.

Dockerfile و محدودیت منابع

برای هر سرویس Dockerfile مستقل بر پایه python:3.12-slim نوشته شد. وابستگی‌ها به نسخه مشخص FastAPI 0.116.1 و Uvicorn 0.35.0 محدود شدند. هر سرویس فقط یک worker دارد و log access تکراری Uvicorn غیرفعال شده است.

محدودیت منابع stack سرویس‌ها

جزء	سقف RAM	سقف CPU	سقف process
match-service	160 MB	0.50	۱۲۸
team-service	160 MB	0.50	۱۲۸
stadium-service	160 MB	0.50	۱۲۸
Nginx	64 MB	0.25	۶۴

```
$ docker compose ps
NAME                SERVICE            STATUS            PORTS
code-match-service-1 match-service      Up (healthy)      8000/tcp
code-nginx-1         nginx              Up (healthy)      0.0.0.0:8080→80/tcp
code-stadium-service-1 stadium-service    Up (healthy)      8000/tcp
code-team-service-1  team-service       Up (healthy)      8000/tcp

$ docker compose exec nginx nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

$ docker stats --no-stream
NAME                CPU %    MEM USAGE / LIMIT  PIDS
code-nginx-1         0.00%    5.35MiB / 64MiB    2
code-team-service-1  0.13%    32.60MiB / 160MiB  2
code-match-service-1 0.08%    34.44MiB / 160MiB  2
code-stadium-service-1 0.14%    33.02MiB / 160MiB  2
```

شکل ۲: اجرای موفق کانتینرها، تست کانفیگ Nginx و مصرف پایه منابع

تنظیم Nginx

Nginx با یک worker و ۱۰۲۴ اتصال تنظیم شد. سه upstream جداگانه تعریف شدند و مسیرهای زیر به سرویس متناظر فرستاده می‌شوند:

● match-service: به /api/matches

● team-service: به /api/teams

● stadium-service. به /api/stadiums

سه header اصلی با proxy_set_header منتقل می‌شوند. اگر X-Request-ID فرستاده نشود، Nginx شناسه داخلی خود را جایگزین می‌کند؛ کشور پیش‌فرض unknown و سناریوی پیش‌فرض normal است. access log با escape=json و به شکل JSON Lines در data/nginx/nginx_access.log نوشته می‌شود.

برای کاهش I/O، بافر ۶۴ کیلوبایتی و flush یک‌ثانیه‌ای برای access log در نظر گرفته شد. همین موضوع هنگام پاک‌سازی اجرای Debug مهم بود و در بخش خطاهای حین اجرا توضیح داده شده است.

```
$ curl -H 'X-Request-ID: manual-001' \
-H 'X-Client-Country: Iran' \
-H 'X-Scenario: normal' \
'http://localhost:8080/api/teams?name=Argentina'
HTTP/1.1 200 OK
{"name":"Argentina","country_code":"ARG","confederation":"CONMEBOL"}

$ curl ... '/api/matches?date=not-a-date'
HTTP status: 400

$ curl -H 'X-Scenario: server_error' ... '/api/stadiums?name=Dallas+Stadium'
HTTP status: 500

$ tail -n 1 data/nginx/nginx_access.log
{"request_id":"manual-001","client_country":"Iran","scenario":"normal",
"path":"/api/teams?name=Argentina","service":"team-service",
"status_code":200,"request_time_sec":"0.019"}

$ tail -n 1 data/service_logs/team_service.log
{"request_id":"manual-001","client_country":"Iran","scenario":"normal",
"service":"team-service","entity_type":"team","entity_value":"Argentina",
"status_code":200,"processing_time_ms":17,"event_type":"team_lookup"}
```

شکل ۳: تست مسیر Nginx، پاسخ‌های ۲۰۰/۴۰۰/۵۰۰ و تطبیق دو سطح لاگ

مولد ترافیک

مولد ترافیک فقط از کتابخانه استاندارد پایتون استفاده می‌کند. برای جلوگیری از مصرف حافظه بالا، ۱۰۰ هزار Future ساخته نمی‌شود؛ فقط تعداد ثابتی worker ایجاد می‌شود و هر worker یک دنباله از شماره درخواست‌ها را پردازش می‌کند. هر worker اتصال HTTP خود را reuse می‌کند و در خطای شبکه فقط یک بار اتصال را بازسازی می‌کند.

ترکیب هدف ترافیک به صورت وزنی تعریف شد تا تفاوت سرویس‌ها و موجودیت‌ها در خروجی دیده شود:

- حدود ۵۰ درصد تیم، ۲۸ درصد مسابقه و ۲۲ درصد ورزشگاه؛
- سناریوهای slow، server_error، invalid، popular، normal؛
- هشت کشور با علاقه غالب متفاوت برای هر کشور؛
- توزیع نامتوازن به نفع آرژانتین، تاریخ ۲۵ ژوئن و ورزشگاه New York New Jersey؛
- شناسه یکتا با قالب req_000001 تا req_100000.

ابتدا ۱۰۰۰ درخواست با ۸ worker اجرا شد. این تست در ۵/۲۸ ثانیه تمام شد و شامل ۹۶۷ پاسخ ۲XX، بیست پاسخ ۴XX و سیزده پاسخ ۵XX بود. سپس منطق پنج Job به صورت محلی روی همین لاگ تست شد. این تست محلی فقط برای debug بود و جای اجرای Hadoop را نگرفت.

اجرای نهایی

اجرای نهایی با ۳۲ اتصال محدود انجام شد. زمان کل ۱۱۹/۷۷ ثانیه و نرخ میانگین ۸۳۴/۹ درخواست در ثانیه بود.

```
$ python3 traffic-generator/generate.py --requests 100000 --workers 32 \
--nginx-url http://localhost:8080
Starting 100,000 requests with 32 workers via http://localhost:8080
Completed: 100,000 in 119.77s (834.9 req/s)
Mean client latency: 35.17 ms
  scenario:invalid: 1,984
  scenario:normal: 91,466
  scenario:popular: 5,072
  scenario:server_error: 1,001
  scenario:slow: 477
  service:match-service: 28,162
  service:stadium-service: 22,219
  service:team-service: 49,619
  status:2xx: 97,015
  status:4xx: 1,984
  status:5xx: 1,001

$ wc -l data/nginx/nginx_access.log data/service_logs/*.log
100000 data/nginx/nginx_access.log
 28162 data/service_logs/match_service.log
 22219 data/service_logs/stadium_service.log
 49619 data/service_logs/team_service.log
200000 total

request_id correlation between gateway and service logs: exact
```

شکل ۴: خروجی اجرای نهایی ۱۰۰هزار درخواست و شمارش فایل‌های لاگ

پس از پایان، تمام خطوط چهار فایل با parser استاندارد JSON بررسی شد. شناسه‌ها در هر فایل یکتا بودند و مجموعه شناسه‌های لاگ Nginx دقیقاً با اجتماع شناسه‌های سه لاگ سرویس برابر بود. حجم فایل Nginx حدود ۲۹ مگابایت و مجموع لاگ سرویس‌ها حدود ۲۸ مگابایت شد.

خطاهای مشاهده‌شده و اصلاح آن‌ها

در اجرای واقعی دو نکته مشخص شد که در تست کاغذی دیده نمی‌شد:

مالکیت فایل‌های mount bind

فایل‌های لاگ در بار اول با مالک root کانتینر ساخته شدند و کاربر میزبان اجازه truncate نداشت. اجرای نهایی فوراً متوقف شد، فایل‌های مشخص از داخل کانتینر truncate شدند و مالکیت پوشه‌های لاگ به کاربر میزبان برگردانده شد. هیچ فایل دیگری حذف نشد.

بافر Nginx پس از توقف ترافیک

چند درخواست در حال اجرا و داده موجود در بافر یک‌ثانیه‌ای، بعد از truncate اولیه وارد فایل شدند. برای اینکه شمار نهایی دقیقاً ۱۰۰ هزار باشد، stack کامل stop شد، فایل‌ها پس از تخلیه بافر صفر شدند و سپس سرویس‌ها دوباره بالا آمدند. check health مسیر جداگانه‌ای دارد و در log access ثبت نمی‌شود؛ بنابراین اجرای نهایی از صفر واقعی شروع شد.

توقف امن generator traffic

در نسخه نخست، KeyboardInterrupt می‌توانست process اصلی را متوقف کند ولی thread‌ها در حال کار برای چند لحظه ادامه دهند. یک Event مشترک و shutdown کنترل‌شده اضافه شد تا هر worker پس از درخواست جاری خارج شود و process پس‌زمینه باقی نماند.

نسخه پایتون image نمونه Hadoop

image ارائه‌شده Python ۵.۳ نصب می‌کرد، در حالی که چند پیام قالب‌بندی mapperها در ابتدا f-string داشتند. قبل از اجرای Job این ناسازگاری پیدا شد، ها f-string به str.format تبدیل شدند و همه map-ها per/reducer داخل خود NameNode با Python ۵.۳ کامپایل شدند.

محدودیت thread در اجرای Spark

در دو اجرای نخست بخش امتیازی، آمار زنده درست در ترمینال دیده شد اما هنگام ذخیره جداگانه هر micro-batch، فراخوانی‌های متعدد فایل‌سیستم محلی تعداد thread/process را به سقف ۲۵۶ رساند. ابتدا JVM با ActiveProcessorCount=2 محدود شد؛ این کار اندازه pool داخلی را کم کرد، ولی نوشتن همزمان خروجی و checkpoint هنوز سربار غیرضروری داشت. چون طبق صورت پروژه نمایش زنده در ترمینال یکی از خروجی‌های قابل قبول است، نوشتن تکراری JSON حذف و checkpoint حفظ شد. اجرای سوم با همان سقف ۲۵۶، حداکثر ۶۲ PID و کد خروج صفر تمام شد. بررسی وضعیت هر دو query نیز به حلقه اجرا اضافه شد تا خطای یکی از جریان‌ها پنهان نماند.

این خطاها بخشی از روند اجرای واقعی پروژه بودند. ثبت آنها نشان می‌دهد خروجی نهایی با اجرای تمیز و قابل تکرار ساخته شده است، نه با ویرایش دستی شمارنده‌ها یا فایل‌های نتیجه.

📖 راه‌اندازی Hadoop و HDFS

Compose نمونه حفظ و با ساختار پروژه هماهنگ شد. کل پروژه روی /project mount می‌شود. نصب Python فقط در NameNode انجام می‌شود، چون DataNode mapper یا reducer اجرا نمی‌کند. محدودیت NameNode برابر ۱۲۰۰ مگابایت و یک CPU، و محدودیت DataNode برابر ۷۰۰ مگابایت و نیم CPU تعیین شد.

پس از healthy شدن دو کانتینر، `hdfs dfsadmin -report` وجود یک DataNode زنده را تأیید کرد. فایل‌های ورودی ابتدا از volume محلی دیده شدند و سپس با `hdfs dfs -put` واقعاً روی HDFS قرار گرفتند.

```
$ docker compose -f hadoop/docker-compose.yml ps
NAME          SERVICE    STATUS          PORTS
datanode      datanode   Up (healthy)    0.0.0.0:9864→9864/tcp
namenode      namenode   Up (healthy)    0.0.0.0:9870→9870/tcp

$ hdfs dfsadmin -report
Configured Capacity: 509943480320 (474.92 GB)
DFS Used: 4096 (4 KB)
Live datanodes (1):
Decommission Status : Normal

$ docker stats --no-stream
NAME          CPU %    MEM USAGE / LIMIT    PIDS
datanode      2.41%    242.3MiB / 700MiB     77
namenode      102.5%   403.1MiB / 1.172GiB   114

Host memory available during Hadoop processing: 2.5 GiB
```

شکل ۵: وضعیت کلاستر، DataNode زنده و مصرف منابع هنگام Streaming Hadoop

📖 پیاده‌سازی پنج Job با Streaming Hadoop

تمام Jobها با یک reducer اجرا شدند تا مصرف حافظه پایین بماند و فایل خروجی نهایی قطعی باشد. برای های‌های map task و reduce سقف ۲۵۶ مگابایت و heap برابر ۱۹۲ مگابایت تعیین شد. اسکریپت `scripts/run_mapreduce.sh` پوشه‌های HDFS را می‌سازد، ورودی‌ها را upload می‌کند، خروجی قبلی هر Job را حذف می‌کند، Jobها را به ترتیب اجرا می‌کند، فایل‌های HDFS را به `outputs/` برمی‌گرداند و در پایان

ها schema را اعتبارسنجی می‌کند.

Job ۱: Cleaning and Parsing

ورودی Job ۱ شامل ۱۰۰ هزار خط Nginx و ۱۰۰ هزار خط سرویس بود. mapper نوع رکورد را از schema تشخیص می‌دهد، وجود تمام فیلدهای اجباری و عددی بودن status/time را بررسی می‌کند و زمان Nginx را از ثانیه به میلی‌ثانیه تبدیل می‌کند. رکوردها با های nginx tag، service، و invalid به reducer می‌روند. reducer داده معتبر را بدون بارگذاری کامل در حافظه عبور می‌دهد.

خروجی واقعی Hadoop شامل ۲۰۰ هزار رکورد بود. پس از جداسازی، ها tag دو CSV صدهزار ردیفی ساخته شد و invalid_logs.csv فقط header داشت؛ خالی بودن بخش داده invalid طبیعی است، زیرا تولیدکننده لاگ schema ثابت دارد.

Job ۲: Aggregation Nginx General

mapper برای هر رکورد سه کلید، endpoint service و scenario تولید می‌کند. مقدار هر کلید شامل شمار کل، موفق، ۴xx و ۵xx و زمان پاسخ است. reducer مجموع، نرخ خطا و میانگین زمان را حساب می‌کند. ۱۰۰ هزار ورودی به ۳۰۰ هزار زوج میانی و در نهایت ۱۱ ردیف آماری تبدیل شد.

Job ۳: Count Request Country–Entity

این Job فقط از cleaned_service_logs.csv استفاده می‌کند. کلید mapper شامل کشور، سرویس، نوع موجودیت و مقدار آن است و reducer تعداد را جمع می‌کند. موجودیت خالی وارد تحلیل محبوبیت نمی‌شود. خروجی نهایی شامل ۲۶۴ ترکیب یکتا بود و به سه فایل تیم، تاریخ و ورزشگاه/شهر تقسیم شد.

Job ۴: Country by Entity Popular

ورودی مستقیم این مرحله خروجی HDFS مربوط به Job ۳ است. mapper داده‌ها را بر اساس کشور گروه‌بندی می‌کند و reducer بیشترین count را انتخاب می‌کند. در تساوی، ترتیب الفبایی برای نتیجه قطعی استفاده می‌شود. سه فایل محبوب‌ترین تیم، روز مسابقه و ورزشگاه هر کشور تولید شد.

Job ۵: Generation Report Final

این مرحله خروجی‌های Job ۲، ۳ و ۴ و فایل metadata پیش‌بینی مسابقات را می‌خواند. reducer تنها یک شیء JSON می‌سازد. مجموع درخواست‌ها از آمار سرویس‌های Job ۲، محبوب‌ترین موجودیت کلی از جمع Job ۳ و محبوب‌ترین تیم هر کشور از Job ۴ به دست می‌آید. اطلاعات فینال پیش‌بینی‌شده نیز از داده ثابت سناریوی سرویس مسابقه وارد metadata شده است.

```
$ docker compose -f hadoop/docker-compose.yml exec -T namenode \
  bash /project/scripts/run_mapreduce.sh
[1/7] Uploading immutable input snapshots to HDFS
[2/7] Job 1 - parsing and cleaning
Map input records=200000
Map output records=200000
Reduce output records=200000
Job job_local658550036_0001 completed successfully
[3/7] Job 2 - general Nginx aggregation
Map input records=100001
Reduce output records=11
Job job_local489911308_0001 completed successfully
[4/7] Job 3 - country/entity request counts
Reduce output records=264
Job job_local1096406958_0001 completed successfully
[5/7] Job 4 - most popular entity by country
[6/7] Job 5 - final summary
Reduce output records=1
Job job_local238000697_0001 completed successfully
[7/7] Validating required outputs
Validated 13 required output files.
MapReduce pipeline completed successfully.
```

شکل ۶: اجرای پشت‌سرهم ها‌Job و تأیید سیزده خروجی الزامی

تحلیل خروجی‌ها

آمار سرویس‌ها

خروجی outputs/job2/service_stats.csv

سرویس	کل	موفق	۴xx	۵xx	میانگین زمان (ms)
match-service	۲۸,۱۶۲	۲۷,۲۹۲	۵۶۳	۳۰۷	۳۷,۲۶۵
stadium-service	۲۲,۲۱۹	۲۱,۵۶۵	۴۵۱	۲۰۳	۳۶,۵۶۴
team-service	۴۹,۶۱۹	۴۸,۱۵۸	۹۷۰	۴۹۱	۳۲,۷۲۳

team-service با ۴۹,۶۱۹ درخواست پرتراфик‌ترین سرویس شد. نرخ خطای match-service برابر ۳/۰۸۹۳ درصد و کمی بیشتر از دو سرویس دیگر بود؛ بنابراین در summary به عنوان سرویس با بیشترین نرخ خطا ثبت شد. میانگین زمان match-service نیز بیشترین مقدار بود و /api/matches به عنوان کندترین

endpoint انتخاب شد.

اثر سناریوی کند

سناریوی slow شامل ۴۷۷ درخواست موفق و میانگین زمان ۸۰۱/۲۲۶ میلی‌ثانیه بود. در مقابل، میانگین سناریوهای عادی و popular نزدیک ۳۱ میلی‌ثانیه ماند. این فاصله نشان می‌دهد داده برای تحلیل re-time sponse تنوع کافی دارد.

محبوبیت موجودیت‌ها

به دلیل توزیع هدفمند، تیم محبوب کلی، Argentina تاریخ پرتکرار 2026-06-25 و ورزشگاه پرتکرار New Stadium Jersey New York شد. با این حال، علاقه تیمی هر کشور مستقل است: برای مثال Iran بیشترین درخواست را برای Iran و Germany بیشترین درخواست را برای Germany داشته است. این نتیجه نشان می‌دهد header کشور واقعاً تا لاگ سرویس منتقل و در های Job ۳ و ۴ استفاده شده است.

```
$ hdfs dfs -ls -R /output
/output/job1/_SUCCESS      /output/job1/part-00000 (25415625 bytes)
/output/job2/_SUCCESS      /output/job2/part-00000 (567 bytes)
/output/job3/_SUCCESS      /output/job3/part-00000 (9138 bytes)
/output/job4/_SUCCESS      /output/job4/part-00000 (1072 bytes)
/output/job5/_SUCCESS      /output/job5/part-00000 (669 bytes)

$ python3 -m json.tool outputs/final/summary.json
{
  "total_requests": 1000000,
  "most_requested_service": "team-service",
  "highest_error_rate_service": "match-service",
  "slowest_endpoint": "/api/matches",
  "most_popular_team_overall": "Argentina",
  "most_requested_match_day_overall": "2026-06-25",
  "most_requested_stadium_overall": "New York New Jersey Stadium",
  "predicted_champion": "Argentina",
  "predicted_final": "France vs Argentina",
  "predicted_final_winner": "Argentina",
  "predicted_final_stadium": "New York New Jersey Stadium"
}
```

شکل ۷: وجود خروجی همه های Job روی HDFS و محتوای summary نهایی

بخش امتیازی Streaming Structured Spark

این بخش پس از پایان کامل Hadoop اجرا شد تا مصرف دو stack روی هم جمع نشود. image رسمی apache/spark:3.5.5 با local[2] اجرا شد. سقف کانتینر ۱۶۰۰ مگابایت RAM و دو CPU حافظه driver برابر ۷۶۸ مگابایت و تعداد partition shuffle برابر دو است. رابط وب Spark نیز غیرفعال شد.

برنامه دو readStream مستقل با schema صریح دارد. جریان اول فایل‌های تازه Nginx را می‌خواند، request_time_sec رشته‌ای را به double و سپس میلی‌ثانیه تبدیل می‌کند و در پنجره‌های ده‌ثانیه‌ای، تعداد درخواست، تعداد خطا، نرخ خطا و میانگین زمان پاسخ را به تفکیک سرویس و endpoint می‌سازد. جریان دوم لاگ سرویس را می‌خواند و محبوب‌ترین تیم هر کشور را در همان پنجره زمانی محاسبه می‌کند. wa-termak سی‌ثانیه‌ای و دو مسیر checkpoint مستقل برای محدودکردن state و ادامه قابل اعتماد جریان در نظر گرفته شد.

اسکرپت export_log_batches.py خطوط منبع را در فایل‌های ۲۰۰تایی می‌ریزد. هر batch ابتدا با پسوند موقت نوشته و سپس با rename اتمیک وارد پوشه stream می‌شود؛ در نتیجه Spark فایل نیمه‌نوشته را نمی‌خواند. در آزمون نهایی این بخش، پنج فایل Nginx و پنج فایل سرویس، هرکدام در مجموع ۱۰۰۰ رکورد، هنگام فعال بودن queryها وارد شدند. مقدارهای epoch صفر پس از رسیدن فایل‌های جدید ظاهر شدند و process با کد صفر خاتمه یافت.

```
$ docker compose -f spark/docker-compose.yml run --rm spark \
  --master 'local[2]' --driver-memory 768m \
  --conf spark.sql.shuffle.partitions=2 spark/streaming_app.py ...
Picked up JAVA_TOOL_OPTIONS: -XX:ActiveProcessorCount=2 -Xss512k
Spark version 3.5.5
Structured Streaming started. Waiting for new JSONL batch files...

$ python3 scripts/export_log_batches.py ... --batch-size 200 --max-lines 1000
exported 1000 nginx lines in 5 new files
exported 1000 service lines in 5 new files

== LIVE GATEWAY WINDOW | epoch 0 ==
window      service      endpoint      requests  errors  avg_ms  error_rate
16:05:00--16:05:10  team-service  /api/teams    493       11     22.882  2.231%
16:05:00--16:05:10  match-service /api/matches  283        7     28.360  2.473%
16:05:00--16:05:10  stadium-service /api/stadiums 224        3     28.237  1.339%

== LIVE POPULAR TEAM BY COUNTRY | epoch 0 ==
Argentina → Argentina (111)  Iran → Iran (76)
Brazil → Brazil (73)         Japan → Japan (83)
Canada → Canada (75)         Mexico → Mexico (72)
Germany → Germany (64)       USA → USA (80)

$ docker stats --no-stream
NAME                                MEM USAGE / LIMIT   CPU    PIDS
spark-spark-run-e953f037fbbf        475.1MiB / 1.562GiB  200.88%  62

process finished with exit code 0
```

شکل ۸: خروجی زنده Spark پس از ورود های batch جدید و مصرف منابع اجرای نهایی

کنترل مصرف CPU و RAM

کنترل منابع فقط در فایل Compose نبود و در زمان اجرا نیز اندازه‌گیری شد:

- در حالت idle مجموع RAM چهار کانتینر API حدود ۱۰۶ مگابایت بود؛
- زیر بار ۳۲ اتصال، Nginx حدود ۶/۳ مگابایت و سرویس‌ها حدود ۵۰، ۳۹ و ۳۷ مگابایت مصرف کردند؛

- generator traffic فقط ۳۲ worker ثابت داشت و فهرست ۱۰۰ هزار Future در حافظه ساخت؛
- API و Hadoop همزمان اجرا نشدند؛ فایل‌های لاگ روی میزبان باقی ماندند و بعد از stop سرویس‌ها Hadoop شروع شد؛
- در حین Job ۱، NameNode همراه process پردازش حدود ۴۰۳ مگابایت و DataNode حدود ۲۴۲ مگابایت RAM داشت؛
- در همان لحظه حدود ۲/۵ گیگابایت RAM سیستم available بود و هیچ کانتینری به سقف حافظه نرسید.
- Hadoop پیش از Spark خاموش شد؛ Spark در اجرای موفق حدود ۴۷۵ مگابایت RAM، دو هسته CPU و ۶۲ PID مصرف کرد.

آزمون‌ها و معیار پذیرش

جمع‌بندی آزمون‌های انجام‌شده

نتیجه	آزمون
موفق	کامپایل نحوی سه سرویس و همه mapper/reducer
موفق	اعتبارسنجی docker compose config
موفق	تست t-nginx داخل image واقعی
healthy	check health چهار کانتینر API
موفق	پاسخ دستی ۲۰۰، ۴۰۰ و ۵۰۰ از مسیر Nginx
۱۰۰۰ درخواست	اجرای Debug با حداقل ۱۰۰۰ درخواست
دقیقاً ۱۰۰,۰۰۰ درخواست	اجرای نهایی
بدون رکورد خراب	parse تمام خطوط JSON
تطبیق دقیق	یکتایی و تطبیق request ID در دو سطح لاگ
یک DataNode	DataNode زنده در dfsadmin -report
همه موفق	های Job ۱ تا ۵ روی Streaming Hadoop
۱۳ فایل معتبر	اعتبارسنجی فایل‌های خروجی
۲۰۰۰ رکورد، کد خروج صفر	Streaming Structured Spark با ورودی تدریجی

روش اجرای پروژه

سرویس‌ها و ترافیک


```

cd code_phase1/code
docker compose up --build -d
docker compose ps
docker compose exec nginx nginx -t

python3 traffic-generator/generate.py \
  --requests 1000 --workers 8 \
  --nginx-url http://localhost:8080

python3 traffic-generator/generate.py \
  --requests 100000 --workers 32 \
  --nginx-url http://localhost:8080

```

برای شروع یک اجرای تمیز، ابتدا stack متوقف می‌شود و سپس چهار فایل لاگ مشخص truncate می‌شوند؛ فایل زنده نباید با rm حذف شود.

Streaming Hadoop

```

docker compose down
docker compose -f hadoop/docker-compose.yml up -d
docker compose -f hadoop/docker-compose.yml ps

docker compose -f hadoop/docker-compose.yml exec -T namenode \
  bash /project/scripts/run_mapreduce.sh

python3 -m json.tool outputs/final/summary.json
docker compose -f hadoop/docker-compose.yml down

```

Streaming Structured Spark

در یک ترمینال جریان اجرا می‌شود و در ترمینال دوم های batch تازه وارد می‌شوند:

```

./scripts/run_spark_demo.sh

python3 scripts/export_log_batches.py \
  --source data/nginx/nginx_access.log \
  --output data/stream/nginx --prefix nginx \
  --batch-size 200 --max-lines 1000 --delay 0.4

python3 scripts/export_log_batches.py \
  --source data/service_logs/team_service.log \
  --output data/stream/services --prefix team \

```



```
--batch-size 200 --max-lines 1000 --delay 0.4
```

ساختار فایل‌های تحویلی

```
code_phase1/code/
|-- docker-compose.yml
|-- nginx/{nginx.conf,proxy_params}
|-- match-service/{main.py,Dockerfile,requirements.txt}
|-- team-service/{main.py,Dockerfile,requirements.txt}
|-- stadium-service/{main.py,Dockerfile,requirements.txt}
|-- traffic-generator/generate.py
|-- hadoop/{docker-compose.yml,hadoop.env,init.sh}
|-- mapreduce/job1_parse_clean/{mapper.py,reducer.py}
|-- mapreduce/job2_nginx_aggregation/{mapper.py,reducer.py}
|-- mapreduce/job3_country_entity/{mapper.py,reducer.py}
|-- mapreduce/job4_popular_entity/{mapper.py,reducer.py}
|-- mapreduce/job5_final_report/{mapper.py,reducer.py}
|-- scripts/{run_mapreduce.sh,validate_outputs.py,...}
|-- spark/{docker-compose.yml,streaming_app.py}
|-- scripts/{run_spark_demo.sh,export_log_batches.py}
|-- data/nginx/nginx_access.log
|-- data/service_logs/*.log
`-- outputs/{job1,job2,job3,job4,final}/
```

جمع‌بندی

مسیر کامل پروژه از درخواست کاربر آزمایشی تا summary نهایی پیاده‌سازی و اجرا شد. درخواست‌ها فقط از Nginx عبور کردند، هر درخواست در دو سطح با شناسه مشترک قابل ردیابی است، داده نهایی دقیقاً ۱۰۰ هزار رکورد gateway دارد و تحلیل‌ها با Streaming Hadoop واقعی انجام شدند. استفاده از اسکریپت ساده یا Pandas جایگزین MapReduce نشده است.

محدودیت منابع، اجرای جداگانه، ها، stack تعداد ثابت ها worker و آزمون تدریجی باعث شد پروژه روی سیستم با RAM محدود پایدار بماند. خروجی نهایی در outputs/final/summary.json و تمام خروجی‌های میانی در پوشه‌های مشخص موجود است.

☆ وضعیت نهایی

سه سرویس، Nginx مولد ترافیک، چهار فایل لاگ نهایی، ده mapper/reducer اسکریپت اجرای زنجیره‌ای، دوازده CSV میانی و summary نهایی آماده و آزموده شده‌اند. بخش اجباری فاز اول کامل است و بخش امتیازی Spark نیز با ورودی تدریجی و خروجی زنده اجرا شده است.